

SecFlowCheck — Analyseur de sécurité CI/CD (GitHub Actions & GitLab CI)

1. Description du projet

SecFlowCheck est un outil d'analyse automatisée conçu pour renforcer la **sécurité des pipelines CI/CD** (Continuous Integration / Continuous Deployment) sur les plateformes **GitHub Actions** et **GitLab CI**.

Il vise à détecter les **mauvaises pratiques de configuration**, les **fuites potentielles de secrets**, et les **vulnérabilités dans les workflows**.

En analysant les fichiers YAML des pipelines, SecFlowCheck identifie les risques liés à la configuration, tels que :

- L'utilisation de secrets en clair,
- Les permissions excessives des jobs,
- Les appels non sécurisés à des actions externes,
- Les dépendances non vérifiées,
- L'absence d'étapes de sécurité comme le scanning de code ou la signature des artefacts.

L'outil peut s'intégrer directement dans le pipeline pour fournir un **rapport de conformité et de sécurité** avant chaque déploiement.

2. Objectifs

1. **Automatiser la détection des vulnérabilités CI/CD** dans les pipelines GitHub Actions et GitLab CI.
2. **Assurer la conformité** aux bonnes pratiques de sécurité DevSecOps (principe du moindre privilège, validation des dépendances, contrôle d'accès).
3. **Fournir un rapport visuel** des risques avec un score de sécurité global par pipeline.
4. **Intégrer SecFlowCheck** dans le workflow pour interrompre automatiquement un build non conforme.
5. **Supporter plusieurs langages et environnements** (Java, Python, Node.js, etc.) via des règles adaptatives.

3. Travaux attendus

a. Étude et veille technologique

- Analyse des menaces et vulnérabilités dans les pipelines CI/CD modernes.
- Étude des outils existants (Checkov, Semgrep CI, GitGuardian, Trivy) et identification des manques.

b. Conception du système

- Définition de l'architecture logicielle (modules d'analyse, moteur de règles, interface CLI/Web).
- Schématisation des flux de traitement (lecture YAML → extraction des jobs → analyse → rapport).

c. Développement

- Implémentation du **parseur YAML** compatible GitHub et GitLab.
- Création du **moteur d'analyse statique** avec un ensemble de règles de sécurité (regex, AST, policy engine).
- Génération de rapports JSON/HTML avec niveaux de gravité (critique, majeur, mineur).
- Intégration dans un pipeline CI/CD réel pour démonstration.

d. Tests et validation

- Création de scénarios de tests avec pipelines vulnérables et conformes.
- Évaluation du taux de détection, des faux positifs et du temps d'analyse.

e. Documentation et livrables

- Manuel d'utilisation et guide d'intégration CI/CD.
- Rapport final contenant l'analyse des résultats et recommandations d'amélioration.

4. Résultats attendus

- Un **outil fonctionnel (CLI + API REST)** capable d'analyser automatiquement des pipelines CI/CD.
- Un **tableau de bord de sécurité** affichant les vulnérabilités détectées et le score global.
- Des **règles personnalisables** pour adapter l'analyse selon les politiques internes d'une entreprise.
- Une **intégration continue** avec GitHub et GitLab pour une exécution automatique à chaque commit.

5. Étapes de réalisation :

a. Analyse et cadrage du projet

Objectif : Comprendre le besoin et définir les attentes fonctionnelles.

Étapes :

1. Identifier les menaces courantes dans les pipelines CI/CD (fuites de secrets, permissions excessives, dépendances vulnérables...).
2. Étudier les plateformes ciblées : structure des fichiers .github/workflows/*.yml et .gitlab-ci.yml.
3. Rédiger un **cahier des charges** précisant :

- Les fonctionnalités principales (analyse statique, rapports, scoring, intégration CI).
- Les contraintes (langages supportés, rapidité d'exécution, compatibilité OS).
- Les livrables (outil, documentation, démonstration).

b. Conception du système

Objectif : Définir l'architecture logicielle et les modules.

Étapes :

1. Concevoir le **diagramme d'architecture** (frontend, backend, moteur d'analyse, base de règles).
2. Spécifier les **modules principaux** :
 - Module d'import YAML
 - Moteur d'analyse (règles statiques et signatures)
 - Générateur de rapports (JSON, HTML)
 - Intégration CI/CD (hook ou plugin)
3. Choisir les **technologies** :
 - Backend : Python (Flask / FastAPI) ou Node.js (Express)
 - Analyse : YAML parser, regex, AST, Semgrep/Checkov API
 - Stockage : SQLite ou MongoDB (historique des analyses)
 - Visualisation : React ou Vue.js (tableau de bord des vulnérabilités)

c. Développement du prototype

Objectif : Implémenter une première version fonctionnelle.

Étapes :

1. Créer le parseur YAML pour extraire les jobs, steps, et variables d'environnement.
2. Développer un **moteur de règles** pour :
 - Identifier les secrets exposés.
 - Vérifier les permissions (permissions: write-all, etc.).
 - Signaler l'absence d'étapes de sécurité (lint, test, scan).
3. Générer un **rapport d'analyse** avec score de sécurité (A–E).
4. Créer une **CLI (Command Line Interface)** pour exécuter l'outil localement.

d. Intégration CI/CD

Objectif : Démontrer l'intégration automatique dans un pipeline réel.

Étapes :

1. Ajouter **SecFlowCheck** comme job dans un pipeline GitHub Actions et GitLab CI.
2. Paramétrer des variables :
 - FAIL_ON_CRITICAL=true pour bloquer un build non conforme.
3. Tester sur des projets exemples :
 - Pipelines vulnérables → erreurs détectées.
 - Pipelines sécurisés → build validé.

e. Tests et validation

Objectif : Vérifier la fiabilité et la précision.

Étapes :

1. Créer un **jeu d'essai** avec différents scénarios :
 - Secrets en clair, dépendances non vérifiées, etc.
2. Évaluer :
 - Taux de détection (précision, rappel).
 - Temps d'analyse moyen.
 - Compatibilité multi-langages.
3. Ajuster les règles pour réduire les faux positifs.

f. Interface de visualisation

Objectif : Fournir un tableau de bord clair.

Étapes :

1. Développer une interface Web affichant :
 - Liste des pipelines analysés.
 - Vulnérabilités détectées par gravité.
 - Score global et recommandations.
2. Intégrer des filtres et exports (CSV, PDF).

g. Documentation et livraison

Objectif : Préparer la mise à disposition du projet.

Étapes :

1. Rédiger :
 - Un **manuel utilisateur** (installation, usage, intégration).
 - Un **guide développeur** (structure du code, extension des règles).

- Une **présentation PowerPoint / rapport technique**.
2. Préparer la **démonstration finale** sur un pipeline réel GitHub ou GitLab.